Research Note: Low-Level Design (LLD) Practice & Evaluation Platform

Candidate: Govind Kumar **Date:** October 2026 **Focus:** 2-Day Engineering Prototype

1. The Core Learner Problem

Practicing Data Structures & Algorithms (DSA) follows a well-defined, deterministic loop: a learner implements code against defined unit tests, receives immediate binary execution feedback (Pass/Fail), and optimizes runtime/space complexity.

In contrast, **practicing Low-Level Design (LLD) is fraught with ambiguity**:

- **Subjective & Multi-Faceted Solutions:** Unlike algorithms with a single deterministic output, an LLD problem (e.g., *Parking Lot*, *Elevator System*, *Movie Ticket Booking*) has dozens of valid implementations. An Elevator can leverage a State Pattern or Command Pattern; both can be effective depending on design trade-offs.
- **Evaluation Void:** Learners write class hierarchies and architectural notes, but remain unsure whether:
  - Class responsibilities adhere to the Single Responsibility Principle (SRP) or form "God Classes".
  - Coupling between high-level orchestration and low-level entities is sufficiently loose (Dependency Inversion Principle).
  - Concurrency, thread-safety, and state synchronization were addressed soundly.
- **Absence of an Iterative Learning Loop:** Existing resources treat LLD as static reading. Candidates read pre-baked reference solutions and attempt to memorize class diagrams, failing to develop the iterative architectural muscle needed to refine a design based on targeted critique.

2. Competitive Landscape & Gaps in Existing Tools

Platform / Tool	Typical Workflow	Feedback Provided	Critical Gap / Failure Mode
LeetCode / HackerRank	Run code against unit test suites.	Pass/Fail boolean & runtime latency (ms).	**Completely unsuitable for LLD.** Only evaluates algorithmic output, ignoring object modeling, interfaces, coupling, or design patterns.
Educative / Grokking LLD	Static reading of reference class diagrams and code.	Static text and diagrams.	**Zero interactive practice.** The learner cannot submit variations or receive feedback on their specific trade-offs.
Generic LLMs (ChatGPT / Claude)	Freeform chat prompt with pasted code.	Unconstrained text critique and arbitrary scores (e.g., "8/10").	**Inconsistent and hallucination-prone.** Lacks standardized engineering rubrics; provides ungrounded scores without citing concrete evidence or identifying domain mismatch.

Platform / Tool	Typical Workflow	Feedback Provided	Critical Gap / Failure Mode
GitHub Repositories	Browse curated implementations.	None (static code reference).	Encourages rote memorization rather than active design synthesis or iterative refinement.

3. Key Product Insights & Gaps Identified

1. Multi-Dimensional Rubric Assessment: Feedback cannot be a single arbitrary score. Useful evaluation must dissect designs across standard engineering dimensions: *Single Responsibility & Cohesion (SRP)*, *Abstraction & Loose Coupling (DIP/ISP)*, *Design Patterns & Extensibility (OCP)*, and *Concurrency Safety & State Management*.

2. Evidence-Based Explanations: To be actionable, every critique must cite **concrete evidence** directly from the candidate's code, explain the structural concern, and suggest an actionable improvement for their next iteration.

3. Iterative Learning Loop (Attempt History): The platform must persist historical attempts so learners can compare Attempt #1 with Attempt #2, observing how extracting an interface or adding synchronization directly impacts their scorecard.

4. Product Direction & MVP Scope

Our prototype centers on the complete **Learner Journey**:

Choose Problem → Design Code & Notes → Submit → Explainable Rubric Scorecard → Dynamic UML → Iterate Attempt

- **Lightweight Go Monolith:** High-performance modular architecture using Go and Fiber, booting instantly with zero operational overhead.
- **Hybrid Decoupled Evaluation Pipeline:**
  - *Deterministic Rule Engine:* Prevents cross-problem contamination, evaluates structural keywords, and dynamically generates Mermaid.js UML class diagrams directly from candidate code.
  - *Pluggable Evaluator Interface:* Satisfies **Change Test B**, allowing external LLM evaluators, AST analyzers, or human reviewers to be chained seamlessly.
- **Resilient Submission Lifecycle:** Submissions are persisted immediately with status `EVALUATING` before processing starts, protected by Go context timeouts, preventing candidate data loss upon evaluator failure.